

Thiranex Full-Stack Personal Portfolio Project

Complete Solution Code: Frontend, Backend, Database & Deployment Guide

1. Project Requirements Satisfied

This solution fully complies with the assignment prompt requirements:

- **Frontend:** Standalone HTML, structured modern CSS grid layout, and custom vanilla JavaScript.
- **Backend:** Node.js with Express.js server frameworks.
- **Database:** Cloud MongoDB via Mongoose connection layer.
- **Deployment Prep:** Structured for hosting on Render (Backend) and Netlify/Vercel (Frontend).

2. Directory Tree Structure

```
portfolio-project/  
  ■■■ backend/  
    ■■■ server.js      # Express main entry point  
    ■■■ package.json   # Node dependencies configuration  
  ■■■ frontend/  
    ■■■ index.html     # HTML markup view layer  
    ■■■ style.css       # Responsive styling definitions  
    ■■■ script.js      # Client-side dynamic API fetcher
```

3. Frontend Layer Code

File: frontend/index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>My Portfolio</title>  
  <link rel="stylesheet" href="style.css">  
</head>  
<body>  
  <header>  
    <div class="logo">Developer Portfolio</div>  
  </header>  
  <section id="projects-section">  
    <h2>My Projects</h2>  
    <div id="projects-container" class="projects-grid">  
      <div class="loading">Loading external project records...</div>  
    </div>  
  </section>  
  <script src="script.js"></script>  
</body>  
</html>
```

File: frontend/style.css

```
* { margin: 0; padding: 0; box-sizing: border-box; font-family: sans-serif; }
body { background-color: #f4f7f6; color: #333; padding: 20px; }
header { text-align: center; margin-bottom: 40px; font-size: 28px; color: #007bff; }
.projects-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 20px; }
.project-card { background: #fff; padding: 20px; border-radius: 8px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); border-top: 4px solid #007bff; }
.project-card h3 { margin-bottom: 10px; }
.project-card p { font-size: 14px; color: #666; margin-bottom: 10px; }
.tech-tag { font-size: 12px; color: #007bff; background: #e6f2ff; padding: 2px 6px; border-radius: 4px; }
```

File: frontend/script.js

```
const API_URL = 'http://localhost:5000/api/projects'; // Replace with live URL after deployment
async function fetchProjects() {
  const container = document.getElementById('projects-container');
  try {
    const response = await fetch(API_URL);
    if (!response.ok) throw new Error("Network latency failed.");
    const projects = await response.json();
    container.innerHTML = '';
    projects.forEach(proj => {
      const card = document.createElement('div');
      card.className = 'project-card';
      card.innerHTML = `<h3>${proj.title}</h3><p>${proj.description}</p><span
class="tech-tag">${proj.techStack.join(', ')}</span>`;
      container.appendChild(card);
    });
  } catch (err) {
    container.innerHTML = '<p style="color:red;">Could not load projects. Ensure backend server
is running.</p>';
  }
}
window.addEventListener('DOMContentLoaded', fetchProjects);
```

4. Backend & Database Layer Code

File: backend/server.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const app = express();
app.use(cors()); app.use(express.json());

const MONGO_URI = process.env.MONGO_URI || "mongodb://localhost:27017/portfolio";
mongoose.connect(MONGO_URI).then(() => console.log("DB Connected")).catch(err => console.log(err));

const projectSchema = new mongoose.Schema({ title: String, description: String, techStack: [String]
});
const Project = mongoose.model('Project', projectSchema);

app.get('/api/projects', async (req, res) => {
  const projects = await Project.find();
  res.json(projects);
});

app.listen(5000, () => console.log("Server operational on Port 5000"));
```

5. Setup & Deployment Instruction Guide

Step 1: Local Setup

Run *npm install express mongoose cors dotenv* in your backend folder. Run *node server.js* to host local service instances.

Step 2: Database Storage Setup

Register via cloud console on MongoDB Atlas, copy the unique URI connection string identifier, and populate it into environment system configurations.

Step 3: Web Deployment Platforms

Deploy backend configurations instantly onto **Render**, and push visual frontend folders straight into **Netlify/Vercel** repositories.